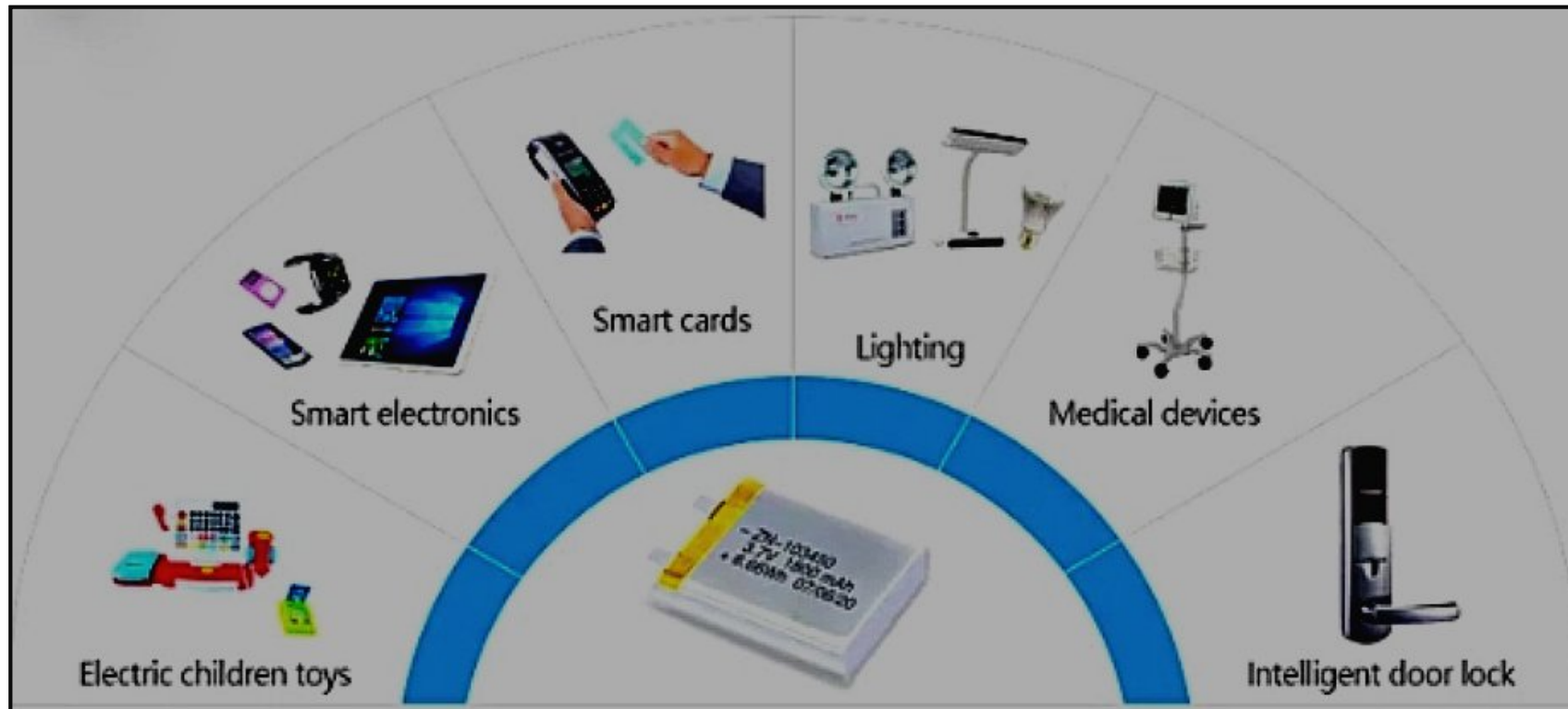


Importancia de los sleep modes



Modos de
operación

Active: El radio está encendido. El chip puede recibir, transmitir o escuchar.

Modem-sleep mode: CPU operacional. Wi-Fi/Bluetooth deshabilitados por momentos.

Light-sleep mode: CPU pausado. La memoria RTC, los periféricos RTC, así como el coprocesador ULP, están en funcionamiento.

Deep-sleep mode: Solo están activos la memoria RTC, los periféricos RTC y el coprocesador ULP. Los datos de conexión Wi-Fi y Bluetooth se almacenan en la memoria RTC. Al despertar, realiza un reset.

RTC: Reloj en
tiempo real

ULP: Ultra Low
Power Coprocessor

Modo	Consumo típico
Activo (radio funcionando)	95 - 240 mA
Modem-sleep	20 - 68 mA
Light-sleep	0.8 mA
Deep-sleep	10 - 150 μ A

Modem-sleep mode

Disponible en modo estación.

```
esp_wifi_set_mode(WIFI_MODE_STA);  
esp_wifi_start();
```

```
/* Configurar el Modem-sleep mode */  
esp_wifi_set_ps(Modo_de_ahorro);
```



`enum wifi_ps_type_t`

Values:

`enumerator WIFI_PS_NONE`

No power save

`enumerator WIFI_PS_MIN_MODEM`

Minimum modem power saving. In this mode, station wakes up to receive beacon every **DTIM period**

`enumerator WIFI_PS_MAX_MODEM`

Maximum modem power saving. In this mode, interval to receive beacons is determined by the `listen_interval` parameter in `wifi_sta_config_t`

DTIM: Delivery Traffic
Indication Message

Light-sleep mode

- La mayor parte del sistema entrará en un estado de suspensión temporal.
- El sistema regresa a modo normal rápidamente cuando ocurre el evento para despertar.
- El CPU está suspendido, no ejecuta código.
- Periféricos como GPIOs, touch pins y timers pueden despertar al ESP32.
- Memoria RTC RAM activa.

RTC RAM

Memoria RAM (8KB) que puede retener los datos cuando el ESP32 está deep sleep o light sleep.

Light-sleep mode despertar por Timer

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_sleep.h"
#include "esp_log.h"
#include "esp_timer.h"
#include "freertos/event_groups.h"

#define TAG "LightSleepTimer"

#define LIGHT_SLEEP_TIME_SEC 10
#define CTN_START_LIGHT_SLEEP 5

EventGroupHandle_t event_group;
const int sensors_ready = BIT0;
static uint16_t light_sleep_cnt = 0;
```

```
void app_main() {
    event_group = xEventGroupCreate();
    xTaskCreate(control_sensores, "sensores", 2048, NULL, 10, NULL);
    xTaskCreate(sys_control, "sys_control", 2048, NULL, 10, NULL);
}
```

```

void control_sensores(void *params)
{
    uint8_t cnt = 0;
    while (true)
    {
        ESP_LOGI(TAG, "Sensores leidos. Cnt: %d. Tiempo: %lld",
            cnt++, esp_timer_get_time());

        if (cnt == CTN_START_LIGHT_SLEEP)
        {
            cnt = 0;
            xEventGroupSetBits(event_group, sensors_ready);
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void sys_control(void *params)
{
    while (true)
    {
        xEventGroupWaitBits(event_group, sensors_ready,
            true, true, portMAX_DELAY);
        light_sleep_cnt++;
        ESP_LOGI(TAG, "Numero de veces en light sleep: %d", light_sleep_cnt);

        /* Configura que se despierte después de 10 segundos (indicado en microsegundos) */
        esp_sleep_enable_timer_wakeup(LIGHT_SLEEP_TIME_SEC * 1000000);

        ESP_LOGI(TAG, "Entrando en light sleep mode por %d segundos...",
            LIGHT_SLEEP_TIME_SEC);

        esp_light_sleep_start(); /* Light sleep */

        ESP_LOGI(TAG, "Despertado de light sleep");
    }
}

```


Activo



Light
sleep



Light-sleep mode despertar por GPIO

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "esp_sleep.h"
#include "esp_log.h"
#include "driver/gpio.h"
#include "driver/rtc_io.h"
#include "esp_timer.h"
#include "freertos/event_groups.h"

#define WAKEUP_PIN GPIO_NUM_4
#define TAG "LightSleepGPIO"
#define CTN_START_LIGHT_SLEEP 5

EventGroupHandle_t event_group;
const int sensors_ready = BIT0;
static uint16_t light_sleep_cnt = 0;
```

```
void app_main()
{
    gpio_reset_pin(WAKEUP_PIN);
    gpio_set_direction(WAKEUP_PIN, GPIO_MODE_INPUT);

    event_group = xEventGroupCreate();
    xTaskCreate(control_sensores, "sensores", 2048, NULL, 10, NULL);
    xTaskCreate(sys_control, "sys_control", 2048, NULL, 10, NULL);
}
```

```

void control_sensores(void *params)
{
    uint8_t cnt = 0;

    while (true)
    {
        ESP_LOGI(TAG, "Sensores leídos. Cnt: %d. Tiempo: %lld", cnt++, esp_timer_get_time());

        if (cnt == CTN_START_LIGHT_SLEEP)
        {
            cnt = 0;
            xEventGroupSetBits(event_group, sensors_ready);
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void sys_control(void *params)
{
    while (true)
    {
        xEventGroupWaitBits(event_group, sensors_ready, true, true, portMAX_DELAY);

        /* Configurar light sleep para despertar con un nivel alto en el pin */
        esp_sleep_enable_ext0_wakeup(WAKEUP_PIN, 1);

        ESP_LOGI(TAG, "Entrando en light sleep, esperando un nivel alto en el GPIO %d...",
            WAKEUP_PIN);
        light_sleep_cnt++;
        /* Entrar en light sleep */
        esp_light_sleep_start();

        if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0)
        {
            ESP_LOGI(TAG, "Despertado por un nivel alto en el GPIO %d", WAKEUP_PIN);
        }
        ESP_LOGI(TAG, "Número de veces en light sleep: %d", light_sleep_cnt);
    }
}

```

Deep-sleep mode

- Es uno de los modos de ahorro de energía más profundos.
- El CPU está suspendido, no ejecuta código.
- La mayor parte del sistema entrará en un estado de suspensión.
- Al despertar realiza un reset del procesador. Es decir, después de despertar, ejecuta el código desde `app_main()`,
- Periféricos como GPIOs, touch pins y timers pueden despertar al ESP32.
- Permite guardar datos que persistan al despertar de deep sleep por medio de la memoria RTC RAM.

Deep-sleep mode despertar por Timer

```
#include <stdio.h>
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_sleep.h"
#include "esp_log.h"
#include "driver/rtc_io.h"
#include "esp_timer.h"
#include "freertos/event_groups.h"

#define DEEP_SLEEP_TIME_SEC 10
#define CTN_START_DEEP_SLEEP 5
#define TAG "DeepSleepExample"

EventGroupHandle_t event_group;
const int sensors_ready = BIT0;

/* Variable en RTC RAM para que el valor persista a través de los ciclos de deep sleep */
RTC_DATA_ATTR static uint16_t deep_sleep_cnt = 0;
```

```
void app_main(void)
{
    event_group = xEventGroupCreate();
    xTaskCreate(control_sensores, "sensores", 2048, NULL, 10, NULL);
    xTaskCreate(sys_control, "sys_control", 2048, NULL, 10, NULL);
}
```



```

void control_sensores(void *params)
{
    uint8_t cnt = 0;
    while (true)
    {
        ESP_LOGI(TAG, "Sensores leídos. Cnt: %d. Tiempo: %lld", cnt++, esp_timer_get_time());

        if (cnt == CTN_START_DEEP_SLEEP)
        {
            cnt = 0;
            xEventGroupSetBits(event_group, sensors_ready);
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void sys_control(void *params)
{
    while (true)
    {
        xEventGroupWaitBits(event_group, sensors_ready, true, true, portMAX_DELAY);
        deep_sleep_cnt++;
        ESP_LOGI(TAG, "Número de veces en deep sleep: %d", deep_sleep_cnt);

        /* Configura que se despierte después de 10 segundos (indicado en microsegundos). */
        esp_sleep_enable_timer_wakeup(DEEP_SLEEP_TIME_SEC * 1000000);
        ESP_LOGI(TAG, "Entrando en deep sleep mode por %d segundos...", DEEP_SLEEP_TIME_SEC);

        esp_deep_sleep_start(); /* Deep sleep */
    }
}

```


Activo



Deep sleep



Deep-sleep mode despertar por GPIO

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "esp_sleep.h"
#include "esp_log.h"
#include "driver/gpio.h"
#include "driver/rtc_io.h"
#include "esp_timer.h"
#include "freertos/event_groups.h"

#define WAKEUP_PIN GPIO_NUM_4
#define CTN_START_DEEP_SLEEP 5

#define TAG "DeepSleepGPIO"

EventGroupHandle_t event_group;
const int sensors_ready = BIT0;

/* Variable en RTC RAM para que el valor persista a través de los ciclos de deep sleep*/
RTC_DATA_ATTR static uint16_t deep_sleep_cnt = 0;
```

```
void app_main() {  
    gpio_reset_pin(WAKEUP_PIN);  
    gpio_set_direction(WAKEUP_PIN, GPIO_MODE_INPUT);  
  
    event_group = xEventGroupCreate();  
    xTaskCreate(control_sensores, "sensores", 2048, NULL, 10, NULL);  
    xTaskCreate(sys_control, "sys_control", 2048, NULL, 10, NULL);  
}
```

```
void control_sensores(void *params)
{
    uint8_t cnt = 0;

    ESP_LOGI(TAG, "Número de veces en deep sleep: %d", deep_sleep_cnt);

    while (true)
    {
        ESP_LOGI(TAG, "Sensores leídos. Cnt: %d. Tiempo: %lld", cnt++, esp_timer_get_time());

        if (cnt == CTN_START_DEEP_SLEEP)
        {
            cnt = 0;
            xEventGroupSetBits(event_group, sensors_ready);
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}
```

```
void sys_control(void *params)
{
    while (true)
    {
        xEventGroupWaitBits(event_group, sensors_ready, true, true, portMAX_DELAY);

        /* Configurar deep sleep para despertar con un nivel alto en el pin */
        esp_sleep_enable_ext0_wakeup(WAKEUP_PIN, 1);
        ESP_LOGI(TAG, "Entrando en deep sleep mode");

        if (esp_sleep_get_wakeup_cause() == ESP_SLEEP_WAKEUP_EXT0)
        {
            ESP_LOGI(TAG, "Despertado por un nivel alto en el GPIO %d", WAKEUP_PIN);
        }
        else
        {
            ESP_LOGI(TAG, "Entrando en deep sleep, esperando un nivel alto en el GPIO %d...", WAKEUP_PIN);
        }

        deep_sleep_cnt++;
        esp_deep_sleep_start(); /* Deep sleep */
    }
}
```

Tarea:

- Revisar la documentación de sleep modes en el ESP32:
https://docs.espressif.com/projects/esp-idf/en/v5.3.1/esp32/api-reference/system/sleep_modes.html
- Ejecutar los ejemplos vistos en clase.